

Multi-Agent Systems



28 Oct 2025 – Lecture 1 - Introduction and Prolog Basics

Course Organization

Multi-Agent Systems
Lecture 1, part 1

Introducing the Teachers



Mike Ligthart

- BSc & MSc AI at Radboud University Nijmegen
- PhD in Social AI and Child-Robot Interaction TU Delft / VU
- Assistant Professor Social AI VU

Research Interests

Sustainable and meaningful human-robot interaction.



Introducing the Teachers



Koen Hindriks

1996: PhD on Agent Programming Languages



Utrecht University

2000: Consultant at **accenture**

2005: Assistant Prof. AI **Radboud University Nijmegen**



2006: Assist. & Assoc. Prof. Interactive Intelligence **TU Delft**



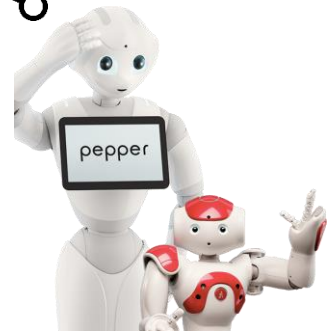
2016: CEO, CIO, Co-Founder Interactive Robotics

2018: Professor Social AI **VU**



Research Interests

- Cognitive agent programming
- Conversational agents
- Social AI
- Social robots
- Socially aware systems



Teaching Assistants



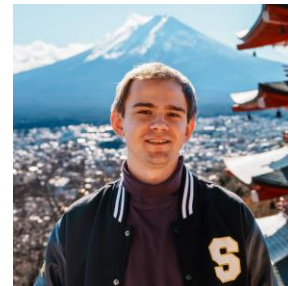
Lazar Gyoshev



Vanshita Kumar



Sofronia Nikolaou



Sławomir Męczyński



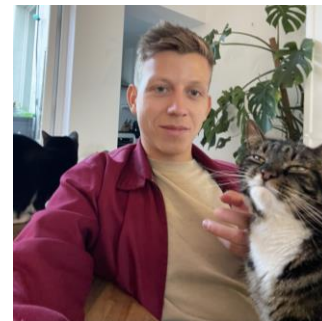
Sofie Veld



Didi Turbatu



Nikolay Pehlivanov



Wouter Schendel

Teaching Assistants



Mateusz Kielan



Snigdha Sharma

Practical Information

Exam

- Thursday Dec-18
- In person digital

Assignment 0

- Individual -> show your TA
- **Not completed = immediate fail**

Assignments 1-3

- Pairs
- Deadline
 - Monday at 23:59h
 - 24h grace period with -1 on grade.
 - **No submission after grace period = immediate fail.**

To pass

- Practical assignments (average) ≥ 5
- Exam ≥ 5
- Weighted average (25% assignment and 75% exam) ≥ 5.5

Practical Information

Grade transfers requests

- Deadline **Today 23.59**
- Assignments and/or exam ≥ 5.0
- See Canvas for schedule details
- You are excused from mandatory attendance, but we recommend you attend anyway.

Practical Information

Pair Programming

- See Canvas for schedule of practicals
- Form pairs with someone from your practical on Canvas (People tab -> Groups)
- Students in groups of 1 will be combined by us into groups of 2.
- Students that did not sign up are assumed to not participate in the assignments.
- **Grade transfers:** we pair you up with someone with the same missing assignment and if we cannot, you will submit it individually.

Practical Information

Practicals

- **Mandatory**
- 2 “slip” days
 - To deal with unforeseen circumstances (e.g. illness)
 - Inform your partner of your absence.
 - You can miss up to two practicals without consequences.
 - If you miss **more than 2** you will **fail the course**.
 - No exceptions.
- Additional practice and Q&A for the assignment.
- Main TA per pair.
- Additional and optional 10-minute TA sessions on request (see Canvas)

Practical Information

Communication (order)

1. Ask you TA.
2. Ask your question on the Canvas Discussion Board.
3. Ask Alexia Muresan

Do not contact us about unforeseen circumstances that lead to you missing the exam or more than 2 practicals.

AI & Intelligent Agents

Multi-Agent Systems
Lecture 1, part 2

Artificial Intelligence =

Computer Vision



Planning & Search



Machine Learning



**Natural
Language
Processing**



This course

**Knowledge
Representation**



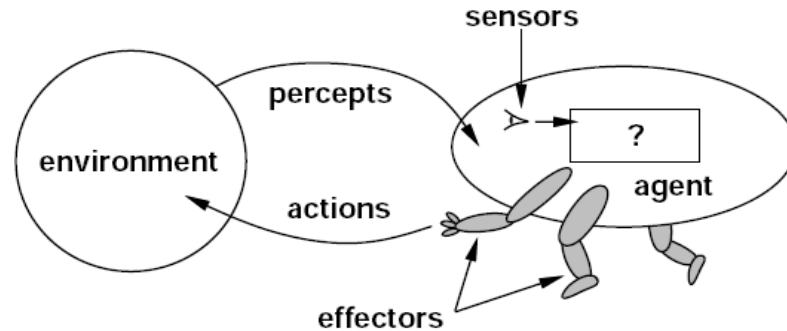
**Agents &
Robotics**



What is an *agent*?

Intelligent Agent Paradigm

An **agent** is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that **environment** through **effectors**.



Internally, an agent is: **agent = architecture + program.**

Core business of AI:
designing agent architectures & programs.

Intelligent Agents

- **Reactive:** ability to receive information and respond
- **Pro-active:** ability to take the initiative
- **Social:** ability to communicate and cooperate
- **Autonomous:** agents control their own processes

(Wooldridge & Jennings, 1995)

Cognitive(-Affective) Agents

A cognitive agent is anything that can be (usefully) **viewed** as a system that has:

- beliefs, desires, goals, intentions, plans, expectations, hopes, fears, joy, ...

(Intentional Stance, Dennett 1987)

Compare:

An agent is anything that can be **viewed** as perceiving its environment through sensors and acting upon that environment through effectors.

Intentional Systems

First-order

- $\text{bel}(p)$: the agent believes that p
- $\text{goal}(p)$: the agent has a goal (or wants) that p

Second-order

- $\text{bel}(A; \text{bel}(B; p))$: A believes that B believes that p
- $\text{bel}(A; \text{goal}(B; p))$: A believes that B wants that p
- $\text{goal}(A; \text{bel}(B; p))$: A wants that B believes that p
- $\text{goal}(A; \text{goal}(B; p))$: A wants that B wants that p

Our notion of “Cognitive Agent”

Agents with the following basic capabilities:

1. **Event processing:** process events like percepts and messages
2. **Knowledge representation:** process events like percepts and messages. It allows maintaining a model of the environment and other agents. (E.g.: use Prolog)
3. **Decision-Making:** agent is able to select an action based on its beliefs, knowledge, and/or goals

Note: We are **not concerned how to transform observations into symbolic representation** (e.g. computer vision). We abstract from this.

Prolog - Basics

Multi-Agent Systems
Lecture 1, part 3

Learning outcomes

1. Student can distinguish between and use the Prolog concepts fact, rule, predicate, clause, query, atom, number, constant, variable, term, compound term, functor, arity, signature.
2. Student can create a syntactically correct Prolog program.
3. Student can create a query and explain how the answer is inferred from the knowledge base.

Prolog compared to other programming languages



- Procedural
 - Object-oriented
 - Functional
 - **Logic-based**
- }
- Imperative
- }
- Declarative
- A **logic-based** programming language, focused on the **description of a problem** (and thus not on how to solve it).
 - Declarative programming vs. imperative/procedural programming
 - Prolog is based on a **different programming paradigm**, and thus requires a **different way of thinking!**

Imperative vs. Declarative

Imagine you walk into your favourite coffee place and that you like to order some coffee

The imperative approach:

1. Enter the coffee shop
2. Queue in the line and wait for the barista asking you for your order
3. Order
4. Yes, for takeaway, please
5. Pay
6. Present your loyalty card to collect points
7. Take your order and walk out

The declarative approach:

- Define rules of coffee place.
- Place an order (=query) : a large latte for takeaway, please

Prolog: general framework



- The aim of prolog is to
 - *represent* knowledge efficiently through facts and rules
 - *retrieve* and *infer* knowledge from this representation, given a query
- Logical principles are needed to enable such inference
 - Truth conditions, connectives, variables, predicates, formula's

Why use prolog?

- Easy to **represent knowledge** through code
- Uses problem solving based on **reasoning**
- Meta-programming:
 - Given a description of a problem, **decides steps itself**
- Ideal for **keeping track of states**



POETS & POETRY: He was a bank clerk in the Yukon before he published “Songs of a Sourdough” in 1907.

Prolog: general framework

- 1: Knowledge base
 - Facts
 - Rules
 - Stored in *.pl* files
- 2: Queries
 - Posted from shell / script
 - ?-

Facts



Nao



Aibo



Pepper



UGV

Knowledge base

```
nao(hero).  
aibo(daisy).  
aibo(cooper).  
pepper(ada).  
ugv(charlie).
```

Some Queries

```
?- nao(hero).  
true.  
?- aibo(charlie).  
false.  
?- aibo(daisy), ugv(charlie).   AND  
true.  
?- aibo(daisy); ugv(cooper).   OR  
true.  
?- daisy.  
ERROR: Unknown procedure:  
daisy/0.
```

Rules



Nao



Pepper



Aibo



UGV

Knowledge base

```
nao(hero).  
aibo(daisy).  
aibo(cooper).  
pepper(ada).  
ugv(charlie).  
  
has_wheels(R) :- pepper(R);ugv(R).
```

$\forall r(\text{pepper}(r) \vee \text{ugv}(r)) \rightarrow \text{has_wheels}(r)$

6 clauses

5 facts, 1 rule

hero = atom, R = variable

Some Queries

```
?- has_wheels(charlie).  
true.  
  
?- has_wheels(hero).  
false.  
  
?- aibo(R).  
R = daisy ;  
R = cooper.  
  
?- has_wheels(ada).  
true ;  
false.
```

Evaluates KB from topdown
and left to right.

Prolog Example

UGV's are typically **not for social interaction**, but more for functional tasks.

Quiz: What is the best way to represent the sociability in our KB?

1. `ugv(R) :- not_social(R).`
2. `social(R) :- nao(R);aibo(R);pepper(R).`
3. `not(social(R)) :- ugv(R).`

Rules

Knowledge base

```
nao(hero).  
aibo(daisy).  
aibo(cooper).  
pepper(ada).  
ugv(charlie).  
  
has_wheels(R) :- pepper(R);ugv(R).  
social(R) :- nao(R);aibo(R);pepper(R).  
  
has_embodiment(R, humanoid) :- nao(R);pepper(R).  
has_embodiment(R, zoomorphic) :- aibo(R).  
has_embodiment(R, machinelike) :- ugv(R).
```



Nao



Pepper



Aibo



UGV

Some Queries

```
has_embodiment(charlie, B).  
B = machinelike.  
  
has_embodiment(R, humanoid).  
R = hero ;  
R = ada.  
  
has_embodiment(daisy, humanoid).  
false.  
  
has_embodiment(R, B).  
R = hero, B = humanoid ;  
R = daisy, B = zoomorphic ;  
...
```


Rules

Knowledge base

```
nao(hero).  
aibo(daisy).  
aibo(cooper).  
pepper(ada).  
ugv(charlie).  
  
has_wheels(R) :- pepper(R);ugv(R).  
social(R) :- nao(R);aibo(R);pepper(R).  
  
has_embodiment(R, humanoid) :- nao(R);pepper(R).  
has_embodiment(R, zoomorphic) :- aibo(R).  
has_embodiment(R, machinelike) :- ugv(R).
```



Nao



Pepper



Aibo



UGV

Some Queries

```
?- has_wheels(R), social(R).  
R = ada.
```

```
?- has_wheels(R), not(social(R)).  
R = charlie.
```

```
?- not(has_wheels(daisy)).  
true.
```

```
?- not(has_wheels(R)).  
false.
```

Prolog - not

- `not(:Goal)` - True if Goal cannot be proven
- `not` is deprecated due to its strong link to logical negation but still provided
- Use `\+` instead.

Prolog Syntax (1)

- :- defines an implication ($A \rightarrow B$ corresponds to $B :- A$).
- , defines a conjunction (in right-hand side of rule)
- ; defines a disjunction (but can also be multiple rules)

An **atom** is one of the following:

1. A sequence of letters, digits, or underscores that *starts with a lower-case letter*, e.g. car, hOUSE_2, father.
2. A sequence of letters, digits or underscores that are *enclosed in single quotes*, e.g. 'car', 'House', 'HOUSE 2'.
3. A string of special characters, e.g. :- or ,

A **number** is any sequence of numbers, possibly containing a dot.

Both atoms and numbers are called **constants**.

Prolog Syntax (2)

A **variable** is a sequence of letters, digits or underscores that *starts with an upper-case letter or an underscore*, e.g. X, Car, HOUSE_2, _father, House.

- The variable _ is called the **anonymous variable**.

A **term** is defined recursively:

1. Each constant and variable is a term.
2. $f(t_1, \dots, t_k)$ is a term if each t_i is a term and f is an atom.
 - This is called a **compound** term with **functor** f and **arity** k , which is often referred to in the notation f/k (**signature**), e.g. has_wheels/1 or has_embodiment/2.
 - A term is **ground** when it contains no variables.

Prolog Syntax (3)

Any constant or compound term is known as a **predicate**.

If immediately followed by a dot (.), it is called a **fact** (knowledge).

Otherwise, it can be a **rule** if:

- Given by $A_0 :- A_1, \dots, A_n$. with $n \geq 0$ and each A_i a predicate.

A_0 is called the **head** of the rule, $A_1, \dots, A_n$ the **body**.

A **clause** is any fact or rule.

- $p(X) :- f(X), r(X)$ (Prolog)
- $f(X) \wedge r(X) \rightarrow p(X)$ (FOL)

Clauses are always **universal**, i.e. here $p(X)$ holds for all X .

Quiz

22	
$p(X) :- f(X), z(Y).$	
$f(x(A), z(B)).$	
Robert	

1. variable, term, predicate
2. constant, term, predicate
3. rule
4. fact, compound term, predicate

Quiz



22	constant, term, predicate
$p(X) :- f(X), z(Y).$	rule
$f(x(A), z(B)).$	fact, compound term, predicate
Robert	variable, term, predicate

1. variable, term, predicate
2. constant, term, predicate
3. rule
4. fact, compound term, predicate

Prolog Syntax (4)

- A **Prolog program** is a finite sequence of clauses.
- The set of all facts and rules that use a certain predicate P is called the **definition** of that predicate. If there is none in a certain program, P is called **undefined** (cannot be queried).
- A **built-in** predicate is defined in SWI Prolog itself, e.g. fail/0, true/0, consult/1, member/2, nth0/3, etcetera.
- A **query** is defined as ‘?- p.’, where p is some predicate, known then as a **goal**.
 - Queries are existential, i.e., ?- p(X) corresponds to $\exists X p(X)$ in FOL.

Learning outcomes

1. Student can distinguish between and use the Prolog concepts fact, rule, predicate, clause, query, atom, number, constant, variable, term, compound term, functor, arity, signature.
2. Student can create a syntactically correct Prolog program.
3. Student can create a query and explain how the answer is inferred from the knowledge base.

Planning:

- Thursday 30 October
 - Work on assignment 0 and 1.
 - Lecture on Prolog unification and search.
 - Preparation: Learn Prolog Now Ch. 1, 2
- Mon November 3
 - Deadline assignment 0.
 - **Not completed = immediate fail.**
- Tuesday November 4
 - Lecture on Prolog Datatypes, Negation and Arithmetic.
 - Preparation: Learn Prolog Now 5.1, 5.2, 10.3